openAG Engine Architecture

Status: Architecture draft — design-only. openAG has zero code yet; AMS is frozen until parity. This document specifies the engine: the one B-ROUND fixpoint, its operators, the neurosymbolic seam, the VoI governor, the component responsibilities + interfaces, confluence, and the AMS fork manifest (what is forked, reused, replaced). It is the companion to `docs/openAG/standard.md` (the data-model contract) and `docs/openAG/tooling.md` (the surface).

Date: 2026-10-12.

---

1. One engine, eight entry points

openAG has exactly one engine — the B-ROUND fixpoint — and eight behaviors (B0–B7) that are entry points into it, not separate pipelines:

behavior	role vs. engine
B0 Stakeholder-needs	opens the continuous ROUND (seed graph).
B1 Extract	drives rounds to promise-fixpoint (top-down + bottom-up).
B2 Author	drives rounds (human promise-fill ceremony).
B3 Re-extract	drives rounds (incremental, identity-preserving).
B4 Represent	reads the converged graph → artifacts.
B5 Generate	reads the graph → code + parity evidence.
B6 Learn	drives rounds (feeds rule changes back).
B7 Gate	is `B-ROUND.gate` run standalone.

Full pipeline detail is in `docs/plans/2026-10-04-openAG-pipelines.md`.

---

2. B-ROUND — the fixpoint

`govern → grow → ground → reconcile → bind → gate`

B-ROUND is a monotone worklist / dataflow framework:

- expanders are the transfer functions (they propose graph deltas);
- **`reconcile`** is the join — a closure operator on the lattice;
- the budget is the widening operator that guarantees termination.

Rounds repeat until no open promise remains within budget. Because `reconcile` is a closure operator, a run is confluent modulo budget: scheduling order changes which gaps appear under a budget cut, never the canonical grounded form (§6).

2.1 The one-step operator

A single promise is discharged by the six-step operator:

`select → propose → ground → admit → reconcile → bind`

The round is the batched operator: `govern` wraps `select` with a schedule, `grow` = `select+propose`, `gate` = batched `admit`. The three-way crosswalk (operator ↔ AMS walk-stage ↔ `model.*` L3 functions) is in the pipelines doc §4 — the existing 13 WalkSpecs + 6-stage runner are the executable prototype of this operator.

3. The neurosymbolic seam

Each operator step is either symbolic or neural, never ambiguously both. The model may invent only in two places and never writes directly — all writes go through symbolic `reconcile`/`gate`.

step	round stage	mode	why
select	govern	symbolic frame + neural VoI (§4)	legality + budget are symbolic; prioritization is neural.
propose	grow	neural	the only place structure is invented.
ground	ground	mixed per-axis	structural→symbolic, semantic→neural; four oracles.
admit	gate	symbolic	schema + constraint predicates.
reconcile	reconcile	symbolic	closure operator (join).
bind	bind	symbolic	commit stable IDs + Generation.

Invariant. Neural steps propose; symbolic steps decide and write. This is what makes confluence hold despite neural components (§6).

4. Neural govern + the VoI economic model

`govern` replaces the earlier deterministic BudgetGovernor with a four-step neural governor:

1. Symbolic legality filter — restrict to the down-only acyclic promise-DAG (the schedulable frontier).
2. Symbolic budget read — owns widening and termination.
3. Neural VoI prioritization — rank the legal frontier by value of information.
4. Symbolic bundle + budget-clamp — bundle discharges; free T1 work auto-drains emergently (not by rule); overflow → `gap:budget-exhausted`.

4.1 VoI

```
VoI = (value × expected confidence gain) ÷ cost
terminate when: top-VoI < cost-floor
```

The VoI model yields three economic gaps: `gap:trivial` (neural triviality judgment), `gap:low-confidence` / `gap:confidence-ceiling` (mixed σ+neural). Seam: triviality → neural, confidence → mixed, cost-floor → symbolic.

4.2 Why neural govern is safe

- Correctness-neutral. Scheduling only changes which gaps surface under a budget cut, not the canonical grounded form (confluence, §6). Govern is an accelerator, not a guarantor.
- Compatibility principle. `govern` schedules only the down-and-local half of the round; `reconcile` owns all up-flows (roll-up + flow-down verification). This keeps the govern DAG acyclic.
- Reproducibility. The neural VoI step runs at temperature 0 so gap-sets are reproducible across runs.

4.3 Risk folds into VoI

Promise risk = `P(won't ground) × consequence` reuses the VoI terms (`consequence` = value, `P` = confidence). No new math, no new entity; the risk register is a VIEW surfaced at gate L2 (§5).

---

5. Gate — two layers

`gate` (and standalone B7) evaluates two layers:

- L1 — symbolic well-formedness (hard). Schema + constraint predicates (the three verification classes). A malformed graph MUST NOT publish.
- L2 — decision-authority go/no-go. The same economic test: `top-VoI < cost-floor` AND critical-path confidence ≥ acceptance. Judgment calls escalate to the human/neural oracle. Provisional roll-ups publish WITH gaps visible — gap-honesty is preserved, not blocked.

---

6. Confluence (modulo budget)

Claim. Two runs of `B-ROUND` on the same inputs under the same budget reach the same canonical grounded form, regardless of scheduling.

Why it holds.

- `reconcile` is a closure operator (idempotent, monotone, extensive) → the join is order-independent.
- Expanders are monotone transfer functions → the worklist computes a least fixpoint.
- The budget is a widening operator → termination is guaranteed; a budget cut only changes which gaps appear, deterministically.
- Neural steps only propose; symbolic `reconcile`/`gate` decide, so non-determinism never reaches the canonical form (temp-0 govern makes the proposed set reproducible too).

Consequence. Confluence is antithetical to trade-space (which needs non-confluence to explore alternatives). Trade-space is therefore a diachronic meta-ROUND (§8), kept out of the synchronic engine.

---

7. Components and interfaces

Gen-0 component ledger (full detail in entity-model §4). Layered with an upward-only dependency rule.

7.1 Substrate layer (extract)

component	responsibility	key interface
SourceScanner	AST-walk the target; emit observable facts (code-scan oracle).	`IF-ScanReport`
PatternDetector	endpoints / clusters (http/cli/message/rpc).	—
Seeder	emit seed nodes with kind-valence open promises.	—

7.2 Kernel layer (model) — the operator

component	responsibility	key interface
ExpansionKernel	run B-ROUND to least-fixpoint; handle `gap:budget-exhausted`.	—
PromiseEngine	compute open promises per node from kind-valence.	`IF-PromiseSpec`
ExpanderRegistry	total `(kind, axis)` → expander (symbolic/neural).	`IF-ExpanderRegistry`
Grounder	mixed per-axis grounding vs. typed evidence; four oracles.	`IF-EvidenceRef`
GateEvaluator	L1 well-formedness + L2 authority; emit typed gaps.	—
Reconciler	closure operator: dedup, θ identity-bind, roll-up, canonicalize.	—
BudgetGovernor	neural govern (4-step); internal-only.	— (internal)
ModelGraph	pure mutable state (anti-god-object); NN-compilable.	`IF-GraphExport`
DecisionCache	memoize expander decisions (incremental re-runs).	— (internal)
IdentityStore	sticky θ-crosswalk; stable IDs across generations.	— (internal)
GenerationStore	publish + version model generations; own lineage.	—
WalkEngine	run declarative WalkSpecs (select→…→emit cards).	`IF-WalkSpec`
SeedStrategy	forward-author vs code-extract policy.	— (internal)

7.3 Projection layer (represent)

component	responsibility	key interface
Slicer	materialize ModelSlice contracts → closed fragments.	`IF-SliceSpec`
ProjectorRegistry	named pure projectors → DiagramSpec.	`IF-ProjectorRegistry`
Renderers	deterministic md/html/svg/zip/ai-context/pipeline-html.	—
Viewer	self-contained HTML lenses (promise-graph, legacy, dashboard).	—

7.4 Adaptation layer (learn)

component	responsibility	key interface
FeedbackJournals	append-only JSONL (gates/drift/test/llm_calls); fail-soft.	`IF-Journal`
LessonStore	deduplicated lesson store.	— (internal)
RuleTuner	tune expander/scheduler rules from lessons; re-enter B-ROUND.	—

7.5 mdd layer + cross-cutting

component	responsibility	key interface
CodeGenerator	generate code; oracle-in-loop convergence.	—
ParityChecker	run the oracle, diff, score parity (runtime oracle).	—
LLMProvider/Router	provider-agnostic routing (policy, budget, fallback, bundling).	`IF-LLMProvider`

---

8. Deferred: the diachronic meta-ROUND

ROUND is a convergent engine covering ~60% of systems engineering. Explicitly deferred (require non-confluence or A6 time):

- Trade-space / decision-management — a diachronic meta-ROUND that runs ROUND once per candidate and selects the winning `Generation`.
- A6 lifecycle — a diachronic reading of the bind θ-crosswalk history.

Both sit outside the synchronic engine and are out of scope for gen-0.

---

9. AMS fork manifest

What openAG does with each of the five existing AMS pipelines (detail: pipelines doc §3):

AMS pipeline	disposition	openAG target
Legacy 10-stage `pipeline/`	REPLACE (drop) — the only one dropped.	— (superseded by B-ROUND).
v3 7-stage `pipeline_v3/`	FORK	B1 Extract (fact source + seeding into the round).
Walk 6-stage `walks/`	REUSE	`CAP-model.traverse` / WalkEngine; operator skeleton.
Lifecycle substrate	FORK	B4 Represent + Projection layer.
CAP-DF ceremony	REUSE	B2 Author (`note → save → author → apply`).

Net: drop one, absorb four. Zero prior AMS effort is lost (zero-loss `$defs` crosswalk, standard §4 / entity-model §11). Genuinely net-new code surface: the fixpoint-kernel internals (ExpanderRegistry, Grounder,

Reconciler, IdentityStore, SeedStrategy), the `axis/valence` promise fields, neural govern, and the `CON-N2` export constraint.

---

10. Build status

Design-only. No code exists. The operators, seam, and components above are the specification a future implementation will satisfy. A TDD build plan is intentionally deferred to a later session.